Лабораторная работа №3

Верификация модели турникета метро в TLA+ / TLC

Касимов Владислав Андреевич

2026

1. Постановка задачи
2. Спецификация на TLA+
3. Свойства
4. Результаты верификации
5. Нетривиальность посылок
6. Анализ невыполненного свойства P4
7. Сравнение nuXmv, SPIN и TLC
Приложение А. Спецификация `Turnstile.tla`
Приложение Б. Конфигурации TLC

1. Постановка задачи

Систему из ЛР1 (турникет в метро, вариант 2) нужно специфицировать на TLA+, сформулировать три свойства на TLA+, проверить их model checker'ом TLC и сравнить подходы nuXmv, SPIN и TLC. Требования к системе и к свойствам те же, что в ЛР1. Описание компонентов приведено в ЛР1, раздел 2.

2. Спецификация на TLA+

Структура

Спецификация имеет стандартный вид:

```
Spec == Init /\ [][Next]_vars /\ Fairness
```

Состояние задают 11 переменных: `user`, `media`, `intr` (среда), `cardRes`, `tokenRes` (защёлки датчиков оплаты), `beam`, `passed` (фотоэлемент: защёлка луча и импульс «проход»), `ctrl`, `gate`, `lamp`, а также вспомогательная (ghost) переменная `crossed`: «после открытия створок кто-то вошёл в проход». `crossed` нужна только для формулировки P3.

`Next` — дизъюнкция 18 действий. Каждое действие описывает один атомарный шаг одного компонента:

Компонент	**Действия**
пассажир (среда)	`Arrive`, `Walk`, `Exit`, `GiveUp`
безбилетник (среда)	`IntrCome`, `IntrLeave`, `IntrSneak`, `IntrExit`
датчики	`CardRead`, `TokenRead`, `PassSense`
контроллер	`Grant`, `Deny`, `DenyDone`, `Close`, `Ready`
актуаторы	`GateMove`, `LampSet`

Семантика — чередование, как в SPIN, но без каналов и счётчиков команд: каждое действие — предикат над `vars` и `vars'`. Актуаторы описаны через целевое значение: `GateMove` разрешён, когда `gate # GateTarget`, где `GateTarget` вычисляется из `ctrl`. Это аналог `next(pos) := open_cmd ? open : closed` из nuXmv с той разницей, что команда исполняется через неопределённое число шагов.

Для каждого действия явно записано, что не изменилось (`UNCHANGED`). Пропуск переменной в `UNCHANGED` — типичная ошибка TLA+: переменная становится произвольной. От этой ошибки страхует инвариант `TypeOK`, который проверяется в каждом прогоне.

Справедливость

```
Fairness ==
  /\ WF_vars(CardRead) /\ WF_vars(TokenRead) /\ WF_vars(PassSense)
  /\ WF_vars(Grant) /\ WF_vars(Deny) /\ WF_vars(DenyDone)
  /\ WF_vars(Close) /\ WF_vars(Ready)
  /\ WF_vars(GateMove) /\ WF_vars(LampSet)
  /\ WF_vars(Walk) /\ WF_vars(Exit) /\ WF_vars(GiveUp) /\ WF_vars(IntrExit)
```

Слабая справедливость задана для всех действий системы и для «реакций» людей: пройти в открытые створки, выйти из прохода, уйти после отказа, выйти безбилетнику. Для `Arrive`, `IntrCome`, `IntrLeave` и

`IntrSneak` справедливость **не задана**. Пассажир не обязан приходить, а безбилетник не обязан проскакивать. Это сохраняет недетерминизм среды: среди поведений есть и такие, где к турникету больше никто не подходит.

Полная спецификация приведена в приложении А.

3. Свойства

В TLA+ нет оператора Until (`U`) и нет `X` как темпорального оператора. Из темпоральных операторов есть только `[]`, `<>`, `~>` (leads-to, `P ~> Q ≡ []( P ⇒ <>Q)`), `WF`/`SF` и свойства шагов `[][A]_v`. Поэтому P1 и P3 переформулированы через инвариант или свойство шагов и живость.

№	Определение	Вид	Соответствие ЛР1
P1	`InvReject == (rejected \/ (ctrl = "denied" /\ lamp # "red")) => gate = "closed"`	инвариант	безопасная часть `closed U red`: от отказа до загорания красного створки закрыты
	`P1Live == rejected ~> lamp = "red"`	живость	часть «красный обязательно загорится» (сильный Until)
P2	`P2 == paid ~> user = "walk"`	отклик	`G(paid → F walk)`
P3	`P3Step == [][gate = "open" /\ gate' = "closed" => crossed]_vars`	свойство шагов	створки не закрываются, пока после открытия никто не вошёл в проход
	`P3Live == gate = "open" ~> gate = "closed"`	живость	открытые створки рано или поздно закроются
P4	`P4 == intr # "sneak"`	инвариант	без оплаты пройти нельзя (**ожидается нарушение**)
L1	`L1 == []<>(ctrl = "idle" /\ gate = "closed")`	живость	система бесконечно часто возвращается в исходное состояние

Почему `φ U ψ` раскладывается так. `p ⇒ (φ U ψ)` эквивалентно конъюнкции двух свойств: `p ⇒ (φ W ψ)` (слабый Until, безопасность) и `p ⇒ ◇ψ` (живость). Слабый Until с фиксированной посылкой выражается инвариантом, если в модели есть переменная, которая отмечает фазу «посылка была, ψ ещё не наступило». Для P1 эту роль играет `ctrl = "denied" ∧ lamp ≠ "red"`: датчик сбросил `cardRes` при `Deny`, но красный ещё не загорелся. Для P3 такой переменной в модели нет, поэтому введена ghost-переменная `crossed`, и проверка идёт через свойство шагов: каждый шаг закрытия обязан произойти после прохода.

4. Результаты верификации

Прогон: `bash lab3-tla/run.sh`. Каждый `cfg/*.cfg` проверяется отдельным запуском TLC (`-workers auto`). Версия: TLC2 2.19 (tla2tools 1.7.4), OpenJDK 17. Во всех прогонах включена проверка тупиков (по

умолчанию) и инвариант `TypeOK`.

Конфигурация	Что проверяется	Сгенерировано / различных состояний	Глубина	Результат
`P1.cfg`	`InvReject`, `P1Live`	322 / 132	18	**OK**
`P2.cfg`	`P2`	322 / 132	18	**OK**
`P3.cfg`	`P3Step`, `P3Live`	322 / 132	18–19	**OK**
`L1.cfg`	`L1`	322 / 132	18	**OK**
`P4.cfg`	`P4`	295 / 129 (остановлен)	17	**нарушен**, трасса из 8 состояний

Каждый запуск занимает около 0.7 с, почти всё это время уходит на старт JVM. Тупиков нет. Полный граф: 132 состояния, 322 перехода.

Состояний 132 против 108 в nuXmv: при чередовании появляются промежуточные состояния, которые синхронный такт проходит за один шаг. Например, контроллер уже `granted`, а створки ещё `closed`, и табло ещё `off`. В SPIN состояний ещё больше (2504), потому что там добавляются счётчики команд процессов и содержимое каналов.

5. Нетривиальность посылок

У TLC, как и у SPIN, нет квантора «существует путь». Поэтому проверяется *отрицание* «посылка выполняется бесконечно часто». Контрпример — это лассо (префикс и цикл), в котором посылка повторяется на каждом обороте.

Конфигурация	Свойство	Свидетель для	Результат	Длина лассо
`W1.cfg`	`<>[](~rejected)`	P1 (`rejected`)	**нарушено**: в цикле есть `cardRes = "fail"`	57 → цикл к сост. 6
`W2.cfg`	`<>[](~paid)`	P2 (`paid`)	**нарушено**: в цикле `tokenRes = "ok"` и `cardRes = "ok"`	29 → цикл к сост. 6
`W3.cfg`	`<>[](gate = "closed")`	P3 (открытие створок)	**нарушено**: `GateMove` открывает створки на каждом обороте	16 → цикл к сост. 5

Цикл в W3 (состояния 5–16): `GateMove(open)` → `Walk` → `LampSet` → `Exit` → `PassSense` → `Close` → `GateMove(closed)` → `Ready` → `LampSet` → `Arrive` → `CardRead` → `Grant` → снова `GateMove(open)`. Это обычный рабочий цикл турникета.

6. Анализ невыполненного свойства P4

Трасса TLC (`logs/P4.log`):

Сост.	Действие	Изменения	Пояснение
1	`Init`		всё свободно
2	`Arrive`	`user = "approach"`, `media = "token_ok"`	пассажир с правильным жетоном
3	`TokenRead`	`user = "wait"`, `tokenRes = "ok"`	жетон принят
4	`Grant`	`ctrl = "granted"`, `tokenRes = "none"`	контроллер разрешил проход
5	`IntrCome`	`intr = "behind"`	безбилетник пристроился сзади
6	`GateMove`	`gate = "open"`	створки открылись
7	`Walk`	`user = "walk"`, `beam = TRUE`, `crossed = TRUE`	пассажир вошёл в проход
8	`IntrSneak`	**`intr = "sneak"`**	безбилетник проскочил следом: **нарушение**

Если продолжить трассу, `PassSense` сработает только когда освободятся оба (`user # "walk" /\ intr # "sneak"`), и выдаст один импульс `passed`. Контроллер выполнит `Close`, как после одного прохода.

Причина та же, что в ЛР1 и ЛР2: фотоэлемент различает только «в проходе кто-то есть / никого нет». Обоснование, почему исправление слишком сложно, приведено в ЛР1, раздел 7. Тот факт, что одно и то же нарушение нашли три инструмента с разной семантикой, подтверждает, что оно вызвано устройством системы, а не артефактом моделирования.

7. Сравнение nuXmv, SPIN и TLC

Аспект	nuXmv	SPIN	TLC
Модель вычислений	синхронная: все модули делают шаг на каждом такте	чередование операторов процессов	чередование действий, стандартная семантика TLA
Язык модели	модули, `next(x) := case`	Promela, процессы, каналы	математика: множества, функции, предикаты над `x, x'`
Взаимодействие	общие переменные	каналы и общие переменные	только общие переменные (каналы можно смоделировать последовательностями)
Заикание	нет	нет	**всегда допустимо** (`[Next]_vars`); без `WF` система может «остановиться»
Справедливость	`FAIRNESS` / `JUSTICE`, здесь не понадобилась	`-f`, слабая, по процессам, одна на всю модель	`WF` / `SF` на любые действия, часть спецификации

Аспект	nuXmv	SPIN	TLC
Логика	CTL, LTL	LTL (с `X` и `U`)	подмножество TLA: `[]`, `<>`, `~>`, свойства шагов; **нет `U` и `X`**
Как записан Until (P1)	`A[closed U red]`	`closed U red`	инвариант + `~>`
Как записан момент открытия (P3)	`closed & X open`	`closed && X open` + `-DNOREDUCE`	свойство шагов `gate = "open" /\ gate' = "closed"` и ghost-переменная
«Существует путь»	`EF`, `EG`	отрицание свойства	отрицание свойства
Алгоритм	символьный (BDD)	явный DFS, never claim, вложенный DFS	явный BFS (кратчайшие контрпримеры), для живости — анализ SCC графа
Состояний	108	2504 (без свойства)	132
Время	0.02 с на всё	≈0.05 с на свойство + ≈1 с компиляции	≈0.7 с на конфигурацию (в основном старт JVM)
Контрпример	состояния, лассо	трасса по процессам и строкам	состояния с именами действий и строками модуля, *Back to state N* / *Stuttering*

Выводы о методах.

- TLC проверяет самую абстрактную спецификацию. Модель — формула, и нет ни тактов, ни процессов, ни каналов. Поэтому модель с чередованием получилась почти такой же компактной, как синхронная модель nuXmv: 132 и 108 состояний.
- Цена этой абстракции в выразительности свойств. Нет `U` и `X`, поэтому пришлось раскладывать Until и вводить ghost-переменную. Зато свойства шагов (`[][A]_v`) выражают требования к *переходам* напрямую, а в LTL это делается только через `X`.
- Заикание и явная справедливость — главное отличие TLA+. В nuXmv «остановиться» нельзя, в SPIN несправедливый планировщик отключается одним флагом, а в TLA+ надо явно сказать, какие действия обязаны происходить. Без `WF` система может просто «остановиться» (заикание), и свойства живости нарушаются.
- BFS в TLC даёт кратчайший контрпример: 8 состояний для P4. DFS в SPIN дал трассу глубины 43, но там счёт идёт по операторам, а не по действиям.
- Результаты всех трёх инструментов совпали: P1–P3 и L1 выполняются, P4 нарушается сценарием «паровозик».

Приложение А. Спецификация `Turnstile.tla`

```
---------------------------- MODULE Turnstile ----------------------------
(***************************************************************************)
(* Лабораторная работа №3. Турникет в метро (вариант 2), TLA+ / TLC.       *)
(* Модель с чередованием: каждый шаг - действие одного компонента.        *)
(*   Среда:        Arrive, Walk, Exit, GiveUp (пассажир),                  *)
(*                 IntrCome, IntrLeave, IntrSneak, IntrExit (безбилетник)  *)
(*   Датчики:      CardRead, TokenRead, PassSense                          *)
(*   Управление:   Grant, Deny, DenyDone, Close, Ready                     *)
(*   Актуаторы:    GateMove, LampSet                                       *)
(* Справедливость (WF) задана только для системы и «реакций» людей;      *)
(* приход пассажира и безбилетника справедливостью не ограничен.          *)
(***************************************************************************)

VARIABLES user, media, intr,       \* среда
          cardRes, tokenRes,       \* датчики оплаты (защёлка до чтения контроллером)
          beam, passed,            \* фотоэлемент: защёлка луча, импульс «проход»
          ctrl, gate, lamp,        \* контроллер и актуаторы
          crossed                  \* вспомогательная (ghost): кто-то прошёл с момента
открытия

vars == <<user, media, intr, cardRes, tokenRes, beam, passed, ctrl, gate, lamp, crossed>>

Media      == {"card_ok", "card_empty", "token_ok", "token_bad"}
CardMedia  == {"card_ok", "card_empty"}
TokenMedia == {"token_ok", "token_bad"}

paid     == cardRes = "ok"   \/ tokenRes = "ok"
rejected == cardRes = "fail" \/ tokenRes = "fail"

TypeOK ==
  /\ user     \in {"away", "approach", "wait", "walk"}
  /\ media    \in Media \cup {"none"}
  /\ intr     \in {"none", "behind", "sneak"}
  /\ cardRes  \in {"none", "ok", "fail"}
  /\ tokenRes \in {"none", "ok", "fail"}
  /\ beam \in BOOLEAN /\ passed \in BOOLEAN /\ crossed \in BOOLEAN
  /\ ctrl \in {"idle", "granted", "denied", "closing"}
  /\ gate \in {"closed", "open"}
  /\ lamp \in {"off", "green", "red"}

Init ==
  /\ user = "away" /\ media = "none" /\ intr = "none"
  /\ cardRes = "none" /\ tokenRes = "none"
  /\ beam = FALSE /\ passed = FALSE
  /\ ctrl = "idle" /\ gate = "closed" /\ lamp = "off"
  /\ crossed = FALSE

(* --------------------------- пассажир ------------------------------- *)
Arrive ==   \* подходит к свободному турникету с произвольным носителем
  /\ user = "away" /\ gate = "closed" /\ lamp = "off"
  /\ user' = "approach" /\ media' \in Media
  /\ UNCHANGED <<intr, cardRes, tokenRes, beam, passed, ctrl, gate, lamp, crossed>>

Walk ==     \* створки открыты - входит в проход
  /\ user = "wait" /\ gate = "open"
  /\ user' = "walk" /\ beam' = TRUE /\ crossed' = TRUE
```

```
  /\ UNCHANGED <<media, intr, cardRes, tokenRes, passed, ctrl, gate, lamp>>

Exit ==     \* выходит из прохода
  /\ user = "walk"
  /\ user' = "away" /\ media' = "none"
  /\ UNCHANGED <<intr, cardRes, tokenRes, beam, passed, ctrl, gate, lamp, crossed>>

GiveUp ==   \* увидел красный, забрал носитель и ушёл
  /\ user = "wait" /\ lamp = "red"
  /\ user' = "away" /\ media' = "none"
  /\ UNCHANGED <<intr, cardRes, tokenRes, beam, passed, ctrl, gate, lamp, crossed>>

(* --------------------------- безбилетник ------------------------------ *)
IntrCome  == intr = "none"   /\ intr' = "behind"
             /\ UNCHANGED <<user, media, cardRes, tokenRes, beam, passed, ctrl, gate, lamp,
crossed>>
IntrLeave == intr = "behind" /\ intr' = "none"
             /\ UNCHANGED <<user, media, cardRes, tokenRes, beam, passed, ctrl, gate, lamp,
crossed>>
IntrSneak == \* вплотную за пассажиром («паровозик»)
  /\ intr = "behind" /\ user = "walk"
  /\ intr' = "sneak" /\ beam' = TRUE /\ crossed' = TRUE
  /\ UNCHANGED <<user, media, cardRes, tokenRes, passed, ctrl, gate, lamp>>
IntrExit  == intr = "sneak"  /\ intr' = "none"
             /\ UNCHANGED <<user, media, cardRes, tokenRes, beam, passed, ctrl, gate, lamp,
crossed>>

(* ---------------------------- датчики -------------------------------- *)
CardRead ==
  /\ user = "approach" /\ media \in CardMedia /\ cardRes = "none"
  /\ cardRes' = IF media = "card_ok" THEN "ok" ELSE "fail"
  /\ user' = "wait"
  /\ UNCHANGED <<media, intr, tokenRes, beam, passed, ctrl, gate, lamp, crossed>>

TokenRead ==
  /\ user = "approach" /\ media \in TokenMedia /\ tokenRes = "none"
  /\ tokenRes' = IF media = "token_ok" THEN "ok" ELSE "fail"
  /\ user' = "wait"
  /\ UNCHANGED <<media, intr, cardRes, beam, passed, ctrl, gate, lamp, crossed>>

PassSense == \* луч перекрывался, проход освободился -> импульс «проход»
  /\ beam /\ user # "walk" /\ intr # "sneak" /\ ~passed
  /\ passed' = TRUE /\ beam' = FALSE
  /\ UNCHANGED <<user, media, intr, cardRes, tokenRes, ctrl, gate, lamp, crossed>>

(* --------------------------- контроллер ------------------------------ *)
Grant ==
  /\ ctrl = "idle" /\ paid
  /\ ctrl' = "granted" /\ cardRes' = "none" /\ tokenRes' = "none"
  /\ UNCHANGED <<user, media, intr, beam, passed, gate, lamp, crossed>>

Deny ==
  /\ ctrl = "idle" /\ rejected
  /\ ctrl' = "denied" /\ cardRes' = "none" /\ tokenRes' = "none"
  /\ UNCHANGED <<user, media, intr, beam, passed, gate, lamp, crossed>>

DenyDone == \* красный показан и носитель забран (валидатор видит пустой слот)
  /\ ctrl = "denied" /\ lamp = "red" /\ media = "none"
  /\ ctrl' = "idle"
  /\ UNCHANGED <<user, media, intr, cardRes, tokenRes, beam, passed, gate, lamp, crossed>>
```

```
Close ==
  /\ ctrl = "granted" /\ passed
  /\ ctrl' = "closing" /\ passed' = FALSE
  /\ UNCHANGED <<user, media, intr, cardRes, tokenRes, beam, gate, lamp, crossed>>

Ready ==
  /\ ctrl = "closing" /\ gate = "closed"
  /\ ctrl' = "idle"
  /\ UNCHANGED <<user, media, intr, cardRes, tokenRes, beam, passed, gate, lamp, crossed>>

(* --------------------------- актуаторы ------------------------------ *)
GateTarget == IF ctrl = "granted" THEN "open" ELSE "closed"
LampTarget == CASE ctrl = "granted" -> "green"
                [] ctrl = "denied"  -> "red"
                [] OTHER            -> "off"

GateMove ==
  /\ gate # GateTarget
  /\ gate' = GateTarget
  /\ crossed' = IF GateTarget = "open" THEN FALSE ELSE crossed
  /\ UNCHANGED <<user, media, intr, cardRes, tokenRes, beam, passed, ctrl, lamp>>

LampSet ==
  /\ lamp # LampTarget
  /\ lamp' = LampTarget
  /\ UNCHANGED <<user, media, intr, cardRes, tokenRes, beam, passed, ctrl, gate, crossed>>

(* ------------------------------------------------------------------- *)
Env    == Arrive \/ Walk \/ Exit \/ GiveUp \/ IntrCome \/ IntrLeave \/ IntrSneak \/ IntrExit
System == CardRead \/ TokenRead \/ PassSense \/ Grant \/ Deny \/ DenyDone
          \/ Close \/ Ready \/ GateMove \/ LampSet
Next   == Env \/ System

Fairness ==
  /\ WF_vars(CardRead) /\ WF_vars(TokenRead) /\ WF_vars(PassSense)
  /\ WF_vars(Grant) /\ WF_vars(Deny) /\ WF_vars(DenyDone)
  /\ WF_vars(Close) /\ WF_vars(Ready)
  /\ WF_vars(GateMove) /\ WF_vars(LampSet)
  /\ WF_vars(Walk) /\ WF_vars(Exit) /\ WF_vars(GiveUp) /\ WF_vars(IntrExit)

Spec == Init /\ [][Next]_vars /\ Fairness

(* ============================== свойства ============================== *)
(* P1. Отказ => створки закрыты, пока не загорится красный.               *)
(*     В TLA+ нет U, поэтому  rejected => (closed U red)  записано как     *)
(*     безопасная часть (инвариант для всей фазы отказа) + живость (~>).    *)
InvReject == (rejected \/ (ctrl = "denied" /\ lamp # "red")) => gate = "closed"
P1Live    == rejected ~> lamp = "red"

(* P2. Подтверждённая оплата => пассажир пройдёт.                          *)
P2 == paid ~> user = "walk"

(* P3. Створки не закрываются, пока после открытия никто не прошёл       *)
(*     (свойство шагов), и открытые створки обязательно закроются.         *)
P3Step == [][gate = "open" /\ gate' = "closed" => crossed]_vars
P3Live == gate = "open" ~> gate = "closed"

(* P4. Без оплаты пройти нельзя. ОЖИДАЕТСЯ НАРУШЕНИЕ («паровозик»).       *)
P4 == intr # "sneak"

(* Живость системы: контроллер бесконечно часто свободен.                 *)
```

```
L1 == []<>(ctrl = "idle" /\ gate = "closed")

(* Свидетели нетривиальности посылок: ожидается НАРУШЕНИЕ, контрпример -  *)
(* поведение (лассо), где посылка выполняется бесконечно часто.          *)
W1 == <>[](~rejected)
W2 == <>[](~paid)
W3 == <>[](gate = "closed")
=============================================================================
```

Приложение Б. Конфигурации TLC

`cfg/P1.cfg` (P2, P3, L1 устроены так же):

```
SPECIFICATION Spec
INVARIANT TypeOK
INVARIANT InvReject
PROPERTY P1Live
```

`cfg/P4.cfg`:

```
SPECIFICATION Spec
INVARIANT TypeOK
INVARIANT P4
```

`cfg/W1.cfg` (W2, W3 устроены так же):

```
SPECIFICATION Spec
PROPERTY W1
```